



Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Desarrollo de Aplicaciones Web	Unidad 3	Tema: De JavaScript a TypeScript
	Semana 9	Tutor: Ing. Victoria Castro

Introducción:

En el desarrollo de software moderno, la eficiencia y la robustez son pilares fundamentales. Si bien JavaScript (JS) es el lenguaje que dio vida a la web, su flexibilidad puede convertirse en un arma de doble filo cuando los proyectos crecen en complejidad. Aquí es donde entra TypeScript (TS).

TypeScript no es un lenguaje nuevo que reemplaza a JavaScript; es una capa de seguridad y estructura que se construye sobre él. Para un ingeniero de software, entender la transición de JS a TS es pasar de escribir "scripts" a construir "sistemas arquitectónicos".

Sección 1:

¿Qué es TypeScript? El Concepto de Superset

TypeScript es un lenguaje de programación desarrollado por Microsoft, definido técnicamente como un **Superset** (superserie) de JavaScript. Esto significa que TypeScript contiene todas las funciones de JavaScript y añade características adicionales, principalmente el **tipado estático**.

El proceso de Transpilación

Es vital entender que los navegadores (Chrome, Firefox, Safari) no pueden leer archivos .ts. Por ello, el código TypeScript pasa por un proceso llamado **transpilación**, donde se traduce a código JavaScript estándar (ES5 o ES6) para que pueda ser ejecutado en cualquier entorno.

Sección 2:

Similitudes y Compatibilidad: La base compartida

La ventaja competitiva de TypeScript es su compatibilidad total. Si ya conoces JavaScript, ya conoces el 90% de TypeScript:

- **Misma Lógica:** Los operadores (+, &&, ===), bucles (for, while) y condicionales (if) son idénticos.
- **Acceso al Ecosistema:** Puedes usar cualquier librería de JavaScript dentro de un proyecto de TypeScript.
- **Curva de aprendizaje suave:** Puedes renombrar un archivo de .js a .ts y, en la mayoría de los casos, seguirá funcionando perfectamente, permitiéndote añadir tipos de forma gradual.

Sección 3:

La Diferencia Fundamental: Tipado Estático vs. Dinámico

En JavaScript, el tipado es **dinámico**: una variable puede guardar un número y, una línea después, un texto. Esto es flexible pero peligroso, ya que causa errores que solo se descubren cuando el usuario final usa la aplicación.

TypeScript introduce el **tipado estático**, donde el desarrollador declara explícitamente qué tipo de dato debe contener una variable.

Ejemplo Comparativo:

En JavaScript (Peligro de error):

```
function calcularTotal(precio, cantidad) {
```

```
    return precio * cantidad;
}

// Si alguien pasa un texto por error:
console.log(calcularTotal(10, "2")); // JS intentará convertirlo, pero puede fallar en lógicas complejas.
```

En TypeScript (Seguridad total):

```
function calcularTotal(precio: number, cantidad: number): number {
    return precio * cantidad;
}

// El editor te avisará del error ANTES de ejecutarlo:
calcularTotal(10, "2"); // ERROR: El argumento de tipo 'string' no es asignable al parámetro de tipo 'number'.
```

Sección 4:

Profundización: Interfaces y el "Contrato" de Código

Uno de los conceptos más potentes en la ingeniería con TypeScript son las **Interfaces**. Una interfaz funciona como un "contrato" que define la forma exacta que debe tener un objeto.

¿Por qué es importante? Imagina que trabajas en un equipo de 50 ingenieros. Si todos crean objetos "Usuario" con diferentes propiedades (uno usa name, otro nombre, otro fullName), el código será un caos. Con una interfaz, obligas a todos a seguir el mismo estándar:

```
interface Usuario {
    id: number;
    nombre: string;
    email: string;
    estaActivo: boolean;
}

const nuevoUsuario: Usuario = {
    id: 1,
```

```
nombre: "Ana García",  
email: "ana@example.com",  
estaActivo: true  
}; // Si olvidas 'email', TypeScript no te dejará compilar el código.
```

Importancia en la Ingeniería de Software

El uso de TypeScript se ha convertido en un estándar de la industria por razones estratégicas:

1. **Mantenibilidad a largo plazo:** Es mucho más fácil entender un código escrito hace dos años si los tipos de datos te dicen exactamente qué hace cada función.
2. **Refactorización Segura:** Si necesitas cambiar el nombre de una propiedad en todo el sistema, el compilador de TS te mostrará exactamente cada lugar donde ese cambio afecta, evitando que "rompas" el sistema.
3. **Documentación Automática:** Los tipos e interfaces sirven como documentación que nunca se desactualiza, porque si el código cambia, los tipos deben cambiar.
4. **Productividad (Tooling):** El autocompletado en editores como VS Code es mucho más preciso, lo que permite escribir código más rápido y con menos errores de dedo.

Bibliografía

TypeScript Official Documentation: La referencia definitiva para entender la sintaxis y las actualizaciones del lenguaje.

- Enlace: <https://www.typescriptlang.org/docs/>

MDN Web Docs (Mozilla): Para comprender las bases de JavaScript sobre las cuales se construye TypeScript.

- Enlace: <https://developer.mozilla.org/es/docs/Web/JavaScript>

Standard ECMA-262: La especificación oficial del lenguaje de scripts en el que se basa JavaScript.

- Enlace: <https://www.ecma-international.org/publications-and-standards/standards/ecma-262/>